

Informe de Progreso 3: Desarrollo de Back End — SIP-Edge

Alcance: Arquitectura del backend, modulos implementados, endpoints API, base de datos, integracion LLM, tests, y configuracion de despliegue.

1. Arquitectura General — Estado: COMPLETADO

Stack tecnologico

Componente	Tecnologia	Version
Lenguaje	Python	3.13+
Framework Web	FastAPI	—
ORM	SQLAlchemy 2.x (Declarative Base)	—
Base de Datos	MariaDB (produccion) / SQLite (tests)	11.8.6
Autenticacion	bcrypt + PyJWT	—
Serializacion	Pydantic v2	—
Serial RS485/RS232	pyserial	—
Motor IA	llama.cpp (llama-server)	b9632
Cliente HTTP	httpx	—
Tests	unittest (stdlib)	—

Estructura de directorios

```
sip_edge/
├── src/                                # 23 modulos ~6,500 lineas
│   ├── main.py                        # Punto de entrada, routers, lifespan
│   ├── config.py                      # Modelo de configuracion (8 secciones)
│   ├── database.py                   # Conexion a BD, SessionLocal
│   ├── models.py                     # Modelos SQLAlchemy (8 tablas)
│   ├── auth.py                       # JWT, bcrypt, RBAC, inactividad
│   └── users.py                      # CRUD de usuarios
```

```
| |─ weighings.py          # CRUD de pesajes + hook RS232
| |─ haciendas.py         # CRUD de haciendas y suertes
| |─ scale.py             # Singleton de comunicacion RS485
| |─ rs232.py             # Envio de tramas CSV a PC externo
| |─ sms_service.py       # Envio de SMS + scheduler reportes
| |─ sms_incoming.py      # Dispatcher de SMS entrantes
| |─ emergency_mode.py    # Modo manual de emergencia
| |─ password_reset.py    # Reset de contrasena via SMS
| |─ llm_client.py        # Cliente HTTP para llama-server
| |─ sql_tools.py         # Catalogo 12 herramientas SQL
| |─ anomaly_detector.py  # Deteccion de anomalias 3 capas
| |─ agent_orchestrator.py # Orquestador LLM + tools + SMS
| |─ report_templates.py  # CRUD de plantillas de reporte
| |─ login_page.py        # Pagina HTML de login con modales
| |─ backup.py            # Respaldo de BD y exportacion
| |─ seed.py              # Seed de admin inicial
| |─ __init__.py
|─ tests/                 # 17 archivos ~7,800 lineas
|─ database/migrations/   # 6 migraciones SQL
|─ scripts/
|   └─ backup.py          # Script de respaldo para bkmngr
```

Patrones de disenno

Patron	Uso	Ubicacion
Singleton	Escucha serial RS485, dispatcher SMS	ScaleService , IncomingSmsDispatcher
Inyeccion de dependencias	FastAPI Depends() para BD y auth	Todos los routers
Template Method	Generacion de reportes con metricas seleccionables	ReportTemplateService
Strategy	Herramientas SQL intercambiables	sql_tools.py (12 tools)
Observer/Handler	Dispatcher de SMS entrantes	IncomingSmsDispatcher con 3 handlers

2. Modulos Implementados — Estado: COMPLETADO

2.1 Configuracion del Sistema (Feature #1)

Archivo	src/config.py (348 lineas)
---------	----------------------------

Modelo de configuracion con 8 secciones tipadas mediante dataclasses `frozen=True` :

```
@dataclass(frozen=True)
class SerialPortConfig: path, baudrate, parity, data_bits, stop_bits
@dataclass(frozen=True)
class SessionConfig: timeout_minutes
@dataclass(frozen=True)
class ScaleConfig: timeout_seconds
@dataclass(frozen=True)
class BackupConfig: keep_days, local_dir, usb_mount_path
@dataclass(frozen=True)
class GsmConfig: apn, modem_index, admin_phones
@dataclass(frozen=True)
class SmsConfig: admin_phones, scheduled_reports
@dataclass(frozen=True)
class AgentConfig: llm_url, llm_model, llm_timeout, z_threshold, window_size, window_hours
@dataclass(frozen=True)
class SystemConfig: rs485, rs232, gsm, session, scale, backup, sms, agent
```

Endpoints:

Metodo	Ruta	Auth	Descripcion
GET	/api/config	admin	Obtiene configuracion completa
PUT	/api/config	admin	Actualiza configuracion completa
PUT	/api/setup/scale	admin	Actualiza solo configuracion de bascula
PUT	/api/setup/session	admin	Actualiza solo configuracion de sesion
POST	/api/config/test/	admin	Prueba de conectividad (rs485/rs232/gsm)

Atomicidad en disco: Toda escritura en `config.yaml` usa archivo temporal + `os.replace()` .

2.2 Autenticacion y RBAC (Feature #2)

Archivo	<code>src/auth.py</code> (138 lineas)
---------	---------------------------------------

Metodo	Ruta	Auth	Descripcion
POST	/api/auth/login	Publico	Login con credenciales, retorna JWT + datos usuario

Funcionalidades:

- Hash de contrasena con bcrypt.

- Token JWT con expiracion configurable y claim `sub` (`user_id`), `role` .
- Dependencia `check_inactivity()` para bloqueo por inactividad (timeout configurable).
- Dependencia `require_role(role)` para RBAC.
- Seed de admin inicial en primer arranque.

2.3 Gestion de Usuarios (Feature #3)

Archivo	<code>src/users.py</code> (174 lineas)
---------	--

Metodo	Ruta	Auth	Descripcion
GET	/api/users	admin	Lista todos los usuarios
GET	/api/users/	admin	Obtiene usuario por ID
POST	/api/users	admin	Crea nuevo usuario
PUT	/api/users/	admin	Actualiza usuario
DELETE	/api/users/	admin	Desactivacion logica

Campos del modelo User: `id` , `username` , `password_hash` , `full_name` , `document` , `role` , `is_active` , `phone` , `failed_login_attempts` , `force_password_change` , `reset_pin` , `reset_pin_expires_at` , `created_at` , `updated_at` .

Endpoints de reset de contrasena (Feature #12):

Metodo	Ruta	Auth	Descripcion
POST	/api/auth/verify-reset-pin	Publico	Verifica usuario + PIN, emite reset_token JWT (5min)
POST	/api/auth/complete-reset	Publico (token)	Cambia contrasena con reset_token valido

2.4 Gestion de Haciendas y Suertes (Feature #4)

Archivo	<code>src/haciendas.py</code> (333 lineas)
---------	--

Metodo	Ruta	Auth	Descripcion
GET	/api/haciendas	admin/operator	Lista haciendas activas

Metodo	Ruta	Auth	Descripcion
GET	/api/haciendas/	admin/operator	Obtiene hacienda con suertes
POST	/api/haciendas	admin	Crea hacienda
PUT	/api/haciendas/	admin	Actualiza hacienda
DELETE	/api/haciendas/	admin	Borrado logico si tiene registros
GET	/api/suertes?hacienda_id=X	admin/operator	Suertes por hacienda (cascada)
POST	/api/suertes	admin	Crea suerte vinculada a hacienda
PUT	/api/suertes/	admin	Actualiza suerte
DELETE	/api/suertes/	admin	Borrado logico

2.5 Integracion Serial con Bascula (Feature #5)

Archivo	<code>src/scale.py</code> (217 lineas)
---------	--

ScaleService — Singleton que gestiona la comunicacion serial RS485 con la bascula DINI ARGEO DFWLI-2.

Comando	Funcion	Descripcion
<code>REXT</code>	Lectura completa	Retorna peso neto, tara, estado, unidad
<code>TARE</code>	Tara semiautomatica	Tareo con el peso actual
<code>TMAN</code>	Tara manual	<code>TMANtttttttt</code> — tara con valor especifico
<code>ZERO</code>	Reset de cero	Funcion tecla ZERO
<code>CLEAR</code>	Limpiar tara	Elimina tara en memoria

Caracteristicas:

- Timeout configurable 1-10s (default 3s).
- Escucha asincrona de datos entrantes (boton PRINT fisico de la bascula).
- `DEV_MODE` omite E/S serial.
- `ScaleService` como singleton inicializado en `main.py`.

2.6 Captura de Pesaje (Feature #6)

Archivo	src/weighings.py (217 lineas)
---------	-------------------------------

Metodo	Ruta	Auth	Descripcion
POST	/api/weighings	admin/operator	Crea registro de pesaje + hook RS232 + hook anomalias
GET	/api/weighings	admin/operator	Lista pesajes
GET	/api/weighings/	admin/operator	Obtiene pesaje por ID

Flujo del operador en kiosco:

1. Seleccionar Hacienda + Suerte (carga en cascada).
2. Ingresar tractomula, vagon, guia.
3. 3 pasos de Tara + Lectura: muestra, mineral, vegetal.
4. Confirmar → POST /api/weighings → commit atomico + envio RS232 + deteccion anomalias.

Hook POST-pesaje:

```
# weighings.py (lineas ~120-127)
frame_data = _build_frame_data(record, hacienda, suerte)
_send_rs232_frame(frame_data, record)          # RS232 al PC
_run_anomaly_detection(record, db)             # Deteccion anomalias
db.commit()                                    # enviado_pc + anomaly_log
```

2.7 Transmision RS232 a PC (Feature #11)

Archivo	src/rs232.py (75 lineas)
---------	--------------------------

Funcion	Descripcion
<code>send_frame(frame_data, format="csv", config_path)</code>	Construye trama CSV y envia por RS232

Formato de trama (CSV fijo, 15 campos):

```
Id,Fecha,Hora,Vagon,Guia,Peso_muestra,0,0,0,0,0,0,0,Peso_vegetal,Peso_mineral
```

- Terminacion CRLF.

- Pesos con 3 decimales.
- Abre/cierra puerto en cada envio (no persistente).
- `DEV_MODE` omite E/S serial.

2.8 Servicio de Notificaciones SMS (Feature #7)

Archivo	<code>src/sms_service.py</code> (339 lineas)
---------	--

Funcion	Descripcion
<code>send_sms(phone, message) -> bool</code>	Envia SMS via mmcli (produccion) o log (dev)
<code>send_alert_to_admins(message)</code>	Alerta de seguridad a todos los admins
<code>send_scheduled_report(report_text)</code>	Reporte programado a admins
<code>start_scheduler()</code>	Inicia planificador asincrono
<code>generate_turn_report(db, turn_start, turn_end)</code>	Genera texto de reporte desde BD

Modos de operacion:

- **Produccion:** Ejecuta `mmcli` contra ModemManager + Quectel EC25.
- **Desarrollo (DEV_MODE):** Simula envio mediante log.

Planificador de reportes: Bucle asincrono que verifica cada 30s si debe enviar reportes. Horarios por defecto: 06:00, 14:00, 22:00. En feature #8 se extiende para usar plantillas configurables desde `ReportTemplateService`.

2.9 Dispatcher de SMS Entrantes (Features #9, #12, #8)

Archivo	<code>src/sms_incoming.py</code> (215 lineas)
---------	---

Dispatcher compartido que recibe y encamina SMS entrantes via polling de `mmcli`.

Handlers registrados:

Prioridad	Handler	Comando	Feature
1	emergency_mode	manual on/off , manual on Xh/Xm , manual on EXT	#9
2	password_reset	reset password <username>	#12
3	ai_query	Cualquier otro texto	#8

2.10 Modo Manual de Emergencia (Feature #9)

Archivo	src/emergency_mode.py (771 lineas)
---------	------------------------------------

Sistema completo con 2 orígenes de activación y 11 sub-requisitos (RF-020a a RF-020k).

Metodo	Ruta	Auth	Descripcion
POST	/api/emergency/request	admin/operator	Solicitar modo manual desde kiosco
GET	/api/emergency/admins	admin/operator	Lista admins disponibles como supervisores
GET	/api/emergency/status	admin/operator	Estado actual del modo manual
GET	/api/emergency/history	admin	Historial de solicitudes

Tabla: emergency_mode_log — auditoria completa de solicitudes, activaciones, extensiones, suspensiones y expiraciones.

2.11 Restablecimiento de Contraseña via SMS (Feature #12)

Archivo	src/password_reset.py (383 lineas)
---------	------------------------------------

Metodo	Ruta	Auth	Descripcion
POST	/api/auth/verify-reset-pin	Publico	Valida usuario + PIN, emite reset_token (5min)
POST	/api/auth/complete-reset	Publico (token)	Cambia contraseña, limpia campos reset

Página de login con modales:

Archivo	src/login_page.py (240 lineas)
---------	--------------------------------

Metodo	Ruta	Auth	Descripcion
GET	/login	Publico	Pagina HTML login con modal PIN + modal cambio contraseña
GET	/	Publico	Redirige a /login si no autenticado

Flujo completo:

- 1. Admin envia SMS "reset password f.ramirez" .
- 2. Sistema valida usuario + telefono, genera PIN 4 digitos hasheado, 1h de validez.
- 3. Envia PIN por SMS al analista.
- 4. Analista hace clic en "Olvido su contraseña" en login.
- 5. Modal 1: ingresa usuario + PIN.
- 6. Si OK, reset_token JWT (5min) permite acceder a Modal 2.
- 7. Modal 2: nueva contraseña + confirmacion.

2.12 Sistema de Reporteria y Deteccion de Anomalias (Feature #8)

Modulos creados (5 archivos nuevos):

Archivo	Lineas	Proposito
src/llm_client.py	160	Cliente HTTP para llama-server API
src/anomaly_detector.py	297	Detector de anomalias en 3 capas
src/sql_tools.py	718	Catalogo de 12 herramientas SQL parametrizadas
src/agent_orchestrator.py	324	Orquestador LLM + SQL Tools + respuesta SMS
src/report_templates.py	324	CRUD de plantillas de reporte + generacion SQL

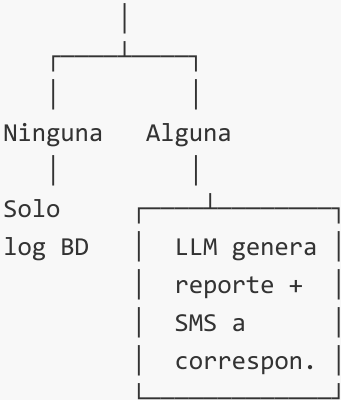
Endpoints:

Metodo	Ruta	Auth	Descripcion
GET	/api/reports/templates	admin	Lista plantillas de reporte

Metodo	Ruta	Auth	Descripcion
POST	/api/reports/templates	admin	Crea plantilla con metricas seleccionables
PUT	/api/reports/templates/	admin	Modifica plantilla
DELETE	/api/reports/templates/	admin	Elimina plantilla
GET	/api/anomalies/status	admin/operator	Estado de deteccion de anomalias
GET	/api/anomalies/history	admin	Historial de anomalias detectadas
POST	/api/agent/query	admin	Consulta directa al agente IA

Sistema de 3 capas de deteccion:

Capa 1 (Z-Score) → $|Z| > 3?$ —Si—> Anomalia
Capa 2 (Ratios) → $\text{vegetal/muestra} > 0.5?$ —Si—> Anomalia
Capa 3 (Temporal) → $\text{tasa_de_cambio} > 50\%?$ —Si—> Anomalia



Catalogo de 12 herramientas SQL:

#	Tool	Parametros
1	get_basic_stats	fecha_inicio, fecha_fin, tipo_material
2	get_percentiles	fecha_inicio, fecha_fin, percentil
3	get_moving_average	window_size, tipo_material
4	get_trend	fecha_inicio, fecha_fin, tipo_material
5	get_breakdown_by_hacienda	fecha_inicio, fecha_fin
6	get_breakdown_by_operator	fecha_inicio, fecha_fin

#	Tool	Parametros
7	get_material_composition	fecha_inicio, fecha_fin
8	get_shift_summary	fecha, turno
9	get_daily_summary	fecha
10	get_custom_period_summary	fecha_inicio, fecha_fin
11	detect_anomalies	window_size, z_threshold
12	check_thresholds	window_size

CPU Pinning:

Configuracion systemd para llama-server:

```
[Service]
ExecStart=/usr/bin/taskset -c 0-2 /usr/local/bin/llama-server \
  -m /home/models/qwen2.5-1.5b-instruct-q4_k_m.gguf \
  -t 3 --threads-batch 3 \
  --host 127.0.0.1 --port 8080
CPUAffinity=0-2
CPUSchedulingPolicy=rr
```

2.13 Sistema de RespalDOS (Feature #10)

Archivo	src/backup.py (138 lineas)
---------	----------------------------

Metodo	Ruta	Auth	Descripcion
GET	/api/backup/status	admin	Ultimos 10 registros de backup_logs
POST	/api/backup/run	admin	Dispara respaldo en background (202 Accepted)

Script externo: scripts/backup.py — ejecutado por usuario bkmngr via cron.

- Volcado mysqldump | gzip con rotacion FIFO de 30 dias.
- Copia a USB con verificacion CRC32.

3. Base de Datos — Estado: COMPLETADO

3.1 Tablas (8 modelos SQLAlchemy)

Tabla	Columnas	PK	FKs	Indices
users	15	id	—	unique(username)
haciendas	6	id	—	unique(codigo)
suertes	6	id	hacienda_id	unique(hacienda_id, codigo_suerte)
weighings	14	id	hacienda_id, suerte_id, usuario_id	—
backup_logs	8	id	—	—
emergency_mode_log	15	id	users (analyst/supervisor)	idx(status, expires_at)
report_templates	7	id	—	—
anomaly_log	8	id	—	—

3.2 Migraciones (6 archivos)

Migracion	Proposito	Feature
000001_add_failed_login_attempts_to_users	Columna failed_login_attempts	#2
000001_create_emergency_mode_log	Tabla emergency_mode_log	#9
000002_add_phone_to_users	Columna phone en users	#7
000003_add_password_reset_fields	Columnas force_password_change, reset_pin, reset_pin_expires_at	#12
000004_create_report_templates	Tabla report_templates	#8
000005_create_anomaly_log	Tabla anomaly_log	#8

4. Integracion LLM — Estado: COMPLETADO

4.1 Cliente HTTP (src/llm_client.py)

Comunicacion con llama-server via HTTP POST a `http://localhost:8080/v1/chat/completions` usando formato compatible con OpenAI API.

Modos de operacion:

Modo	Condicion	Comportamiento
Produccion	DEV_MODE=false	Envia request real a llama-server, procesa tool_calls
Desarrollo	DEV_MODE=true	Retorna respuestas simuladas, sin conexion HTTP

Excepciones:

Excepcion	Contexto
<code>LlamaConnectionError</code>	Error de conexion, timeout, respuesta invalida

4.2 Flujo de Function Calling

1. Recibir pregunta (SMS o API)
 2. Enviar a llama-server con:
 - System prompt + tools definitions (12 tools)
 3. LLM responde con tool_call (nunca numeros directos)
 4. Ejecutar tool SQL con datos reales de BD
 5. Pasar resultado real al LLM para parafrasis
 6. LLM genera respuesta textual
 7. Enviar respuesta (SMS o API response)

5. Procesamiento de SMS (Doble Via) — Estado: COMPLETADO

5.1 Via Saliente

`SMSService` envia SMS via `mmcli` (produccion) o log (desarrollo).

Planificador: Bucle asincrono que verifica cada 30s si debe enviar reportes. Reutiliza `ReportTemplateService` (feature #8) para plantillas configurables.

5.2 Via Entrante

`IncomingSmsDispatcher` en `src/sms_incoming.py` — polling de `mmcli` para leer SMS entrantes y encaminarlos segun el comando:

Comando	Handler	Accion
<code>manual on/off</code>	EmergencyMode	Activa/suspende modo manual
<code>reset password X</code>	PasswordReset	Genera PIN, envia SMS al analista
(cualquier otro texto)	AI Agent	Consulta al LLM con herramientas SQL

6. Endpoints API (Resumen Completo) — Estado: COMPLETADO

Total: 114 endpoints (incluyendo Pydantic schema accessors)

Endpoints funcionales por modulo:

Modulo	Endpoints	Autenticacion
Auth	POST /api/auth/login, POST verify-reset-pin, POST complete-reset	Publico
Config	GET/PUT /api/config, POST test, PUT setup/*	admin
Users	GET/POST/PUT/DELETE /api/users	admin
Haciendas	GET/POST/PUT/DELETE /api/haciendas	admin/operator
Suertes	GET/POST/PUT/DELETE /api/suertes	admin/operator
Weighings	GET/POST /api/weighings, GET /	admin/operator
Emergency	GET/POST /api/emergency/*	admin/operator
Reports	GET/POST/PUT/DELETE /api/reports/templates	admin
Anomalies	GET /api/anomalies/status, /history	admin/operator
Agent	POST /api/agent/query	admin
Backup	GET /api/backup/status, POST /api/backup/run	admin
Health	GET /health	Publico
Pages	GET /login, GET /	Publico

7. Tests — Estado: COMPLETADO

7.1 Resumen

Metrica	Valor
Archivos de test	17
Lineas de test	~7,800
Tests totales	430
Framework	unittest (stdlib)
BD en tests	SQLite en memoria / archivo temporal
Hardware simulado	unittest.mock para serial, mmcli, HTTP

7.2 Distribucion por modulo

Archivo	Tests	Lineas	Cubre
test_emergency_mode.py	—	1,177	Feature #9 (modo manual)
test_password_reset.py	51	1,006	Feature #12 (reset SMS)
test_haciendas.py	—	832	Feature #4 (haciendas/suertes)
test_auth.py	—	591	Feature #2 (auth/RBAC)
test_sms_service.py	—	511	Feature #7 (SMS)
test_backup.py	—	455	Feature #10 (respaldos)
test_users.py	—	443	Feature #3 (usuarios)
test_weighings.py	—	404	Feature #6 (pesaje + RS232)
test_scale.py	—	383	Feature #5 (bascula RS485)
test_config.py	—	336	Feature #1 (configuracion)
test_anomaly_detector.py	17	330	Feature #8 (3 capas)
test_report_templates.py	—	317	Feature #8 (plantillas)
test_sql_tools.py	24	238	Feature #8 (herramientas SQL)
test_agent_orchestrator.py	8	244	Feature #8 (orquestrador)

Archivo	Tests	Lineas	Cubre
test_rs232.py	8	215	Feature #11 (trama CSV)
test_llm_client.py	10	188	Feature #8 (cliente LLM)
test_database.py	—	151	Conexion BD

7.3 Comandos de ejecucion

```
# Todos los tests
docker compose exec backend python -m unittest discover -s tests -v

# Test especifico
docker compose exec backend python -m unittest tests.test_rs232 -v

# Sin Docker (SQLite)
python -m unittest discover -s tests -v
```

8. Configuracion de Despliegue — Estado: COMPLETADO

8.1 Servicio systemd: sip-edge.service

```
[Unit]
Description=SIP-Edge Backend
After=mariadb.service network.target

[Service]
Type=simple
User=sipedge
WorkingDirectory=/home/sipedge/sip_edge
EnvironmentFile=/home/sipedge/sip_edge/.env
ExecStart=/home/sipedge/sip_edge/venv/bin/uvicorn src.main:app \
    --host 0.0.0.0 --port 8000
Restart=always
RestartSec=5
WatchdogSec=30

[Install]
WantedBy=multi-user.target
```

8.2 Variables de entorno (.env)

```
DB_HOST=localhost
DB_NAME=sip_edge
DB_USER=sip_user
```

```
DB_PASSWORD=sip_pass
JWT_SECRET_KEY=<generated>
ADMIN_DEFAULT_PASSWORD=admin
DEV_MODE=false
```

8.3 config.yaml

Archivo de configuracion con 8 secciones: `rs485`, `rs232`, `gsm`, `session`, `scale`, `backup`, `sms`, `agent`. Escritura atomica mediante `tempfile` + `os.replace()`.

8.4 Dependencias (requirements.txt)

```
fastapi, uvicorn, sqlalchemy, pymysql, pydantic, bcrypt, pyjwt, pyserial, httpx
```

9. Verificacion y Health Check — Estado: COMPLETADO

9.1 Health Check

```
# Endpoint publico
curl http://localhost:8000/health
# → {"status": "ok"}

# En EdgeBox remota
curl http://192.168.1.42:8000/health
```

9.2 Verificacion post-despliegue

```
# 1. Estado del servicio
ssh sipedge@192.168.1.42 "sudo systemctl status sip-edge"

# 2. Logs en tiempo real
ssh sipedge@192.168.1.42 "sudo journalctl -u sip-edge -f"

# 3. Tests de hardware
ssh sipedge@192.168.1.42 "cd /home/sipedge/sip_edge && \
    source venv/bin/activate && \
    python -m unittest discover -s tests_hardware -v"

# 4. Smoke test
curl http://192.168.1.42:8000/health
```

10. Resumen de Archivos del Backend

Archivo	Lineas	Proposito
src/main.py	789	Punto de entrada, routers, lifespan
src/sql_tools.py	718	Catalogo 12 herramientas SQL parametrizadas
src/emergency_mode.py	771	Modo manual de emergencia
src/config.py	348	Modelo de configuracion (8 secciones)
src/sms_service.py	339	Envio de SMS y scheduler
src/haciendas.py	333	CRUD de haciendas y suertes
src/report_templates.py	324	CRUD de plantillas de reporte
src/agent_orchestrator.py	324	Orquestador LLM + tools + SMS
src/anomaly_detector.py	297	Deteccion de anomalias 3 capas
src/login_page.py	240	Pagina HTML login con modales
src/models.py	220	Modelos SQLAlchemy (8 tablas)
src/scale.py	217	Singleton comunicacion RS485
src/weighings.py	217	CRUD de pesajes
src/sms_incoming.py	215	Dispatcher de SMS entrantes
src/auth.py	138	JWT, bcrypt, RBAC, inactividad
src/backup.py	138	Respaldo de BD y exportacion
src/llm_client.py	160	Cliente HTTP llama-server
src/users.py	174	CRUD de usuarios
src/password_reset.py	383	Reset de contraseña via SMS
src/rs232.py	75	Envio tramas CSV por RS232
src/database.py	34	Conexion BD y SessionLocal
src/seed.py	28	Seed de admin inicial
	~6,500	Total codigo fuente

11. Conclusion

El desarrollo del backend de SIP-Edge se completo exitosamente con **12 features implementadas** a traves de **23 modulos (~6,500 lineas)** y **17 archivos de test (~7,800 lineas)**.

Aspecto	Resultado
Features	12/12 completadas
Modulos	23 archivos en src/
Endpoints	~30 funcionales
Tests	430 tests, todos pasan en verde
Base de Datos	8 tablas, 6 migraciones
Hardware	RS485 (bascula), RS232 (PC), GSM (SMS duplex)
IA	Qwen 2.5 1.5B con Function Calling, 3 cores dedicados
Seguridad	JWT, bcrypt, RBAC, PIN hasheado, bloqueo por inactividad
Resiliencia	Modo manual de emergencia, respaldos automaticos, watchdog

El sistema esta desplegado en la EdgeBox-RPI-200 (192.168.1.42) con MariaDB 11.8.6, Python 3.13+, llama.cpp con Qwen 2.5 1.5B, y todos los servicios gestionados por systemd.

Documento generado a partir del analisis completo del codigo fuente, tests, migraciones y artefactos del desarrollo SDD.